E-commerce Customer & Orders Analysis

Challenge-Based Trainer Guide | Python Data Analysis | 2 Hours

Dataset note: all records are synthetic. The main technical learning objective is combining related tables without double-counting order revenue.

1. Business scenario

An e-commerce retailer wants to understand which channels and cities generate orders and revenue, which product categories contribute profit, how many orders are cancelled or returned, and whether customers place repeat orders. Students receive three raw tables and must inspect, clean, join, analyze, visualize, and communicate recommendations.

2. Challenge outputs

• A data-quality report for all three tables.

• A documented relationship map: Customers → Orders → Order Items.

• A cleaned analysis table with one row per order and a separate item-level table.

• At least 4 KPIs, 3 charts, and one interactive Streamlit dashboard.

• Three findings and two recommendations supported by metrics.

3. Two-hour schedule

Time	Challenge	Hint / checkpoint
0–10 min	Understand business questions and table relationships.	Identify primary keys: Customer_ID, Order_ID, and the item table's Order_ID.
10–25 min	Import and profile all three CSVs.	Use a loop to inspect shape, info, missing values, and duplicates.
25–45 min	Clean data and validate keys and numeric ranges.	Remove exact duplicates; check quantity > 0, price > 0, discount between 0 and 0.8.
45–70 min	Join tables and calculate KPIs.	Aggregate order items by Order_ID before merging totals into orders.
70–90 min	Build charts using Matplotlib, Seaborn, Plotly.	Monthly trend, category profit/revenue, order status or customer order frequency.
90–115 min	Explore the Streamlit dashboard.	Filter by channel, city, customer segment, and order status.
115–120 min	Present one insight and one recommendation.	State metric, comparison, limitation, and action.

4. Key KPIs

KPI	Definition	Caution
Delivered revenue	Sum of item-level sales for delivered orders.	Do not include cancelled orders as realized revenue.
Order average value	Delivered revenue ÷ number of delivered orders.	Use one row per order in denominator.
Category profit	Sum of line sales minus line costs.	Analyze at item-line grain.
Repeat customer rate	Customers with more than one order ÷ customers in the selected period.	Define the period and customer base clearly.
Return/cancellation rate	Returned or cancelled orders ÷ all orders.	Report statuses separately if business meaning differs.

5. Guided code checkpoints

Import and inspect

```
customers = pd.read_csv("customers_raw.csv")
orders = pd.read_csv("orders_raw.csv")
items = pd.read_csv("order_items_raw.csv")
for name, table in [("customers",customers),("orders",orders),("items",items)]:
    print(name, table.shape)
    print(table.info())
    print(table.isna().sum())
    print("Duplicates:", table.duplicated().sum())
```

NumPy: use median and percentiles to inspect line sales and unusual values.

```
values = pd.to_numeric(items["Line_Sales_EGP"], errors="coerce").dropna().to_numpy()
print(np.median(values), np.percentile(values, 95))
```

Join safely: the most important hint

```
item_totals = items.groupby("Order_ID", as_index=False).agg(
```

```
    Revenue=("Line_Sales_EGP", "sum"),
    Profit=("Line_Profit_EGP", "sum"),
    Units=("Quantity", "sum")
)
orders_analysis = orders.merge(item_totals, on="Order_ID", how="left")
orders_analysis = orders_analysis.merge(
    customers[["Customer_ID", "City", "Customer_Segment"]],
    on="Customer_ID", how="left"
)
```

6. Library mapping

• NumPy: median and percentiles for item values and numeric validation.

• Pandas: cleaning, date parsing, merge/join, groupby, aggregation, customer/order features.

• Matplotlib: static revenue by channel or category chart.

• Seaborn: distribution of line profit by category or correlation heatmap.

• Plotly: interactive monthly revenue trend and city/channel comparisons.

• Streamlit: filters, KPI cards, order-status view, category performance, data-quality panel.

7. Discussion prompts and answer cues

Why use three tables? To represent entities at their natural grain and avoid repeating customer details in every item line.

Why aggregate items before merging to orders? One order can contain multiple item rows; joining unaggregated lines to orders multiplies order-level values.

Should cancelled orders count as revenue? Usually not as realized sales; separate order intent from delivered sales and define return handling.

Which channel is best? Compare order volume, delivered revenue, profit, status mix, and customer behavior—not revenue alone.

What is a good recommendation? Name the action, supporting metric, expected benefit, and a follow-up metric.

8. Run the dashboard

• Install: pip install -r requirements.txt

• Run: streamlit run app.py

• Keep all three CSV files beside app.py.

• Demonstrate sales channel, city, customer segment, and order status filters.

Instructor note: reveal hints in stages. The dashboard and starter script are reference solutions; let students attempt the joins and KPI definitions before showing them.